PD and PID Control

Contents

1. Overview . . . 2
2. Why turning swings: force, torque and the moment of inertia . . . 3
   - 2.1 Driving: force sets the acceleration . . . 4
   - 2.2 Turning: torque, angular acceleration and the moment of inertia . . . 4
   - 2.3 Why the heading swings: one adding-up more . . . 7
3. Proportional control: twisting in proportion to the error . . . 8
   - 3.1 The rule, and why it overshoots . . . 8
   - 3.2 A spring with nothing to stop it . . . 9
4. Derivative action: braking before the target . . . 10
   - 4.1 Look at how fast the error shrinks . . . 10
   - 4.2 How much derivative . . . 11
5. Integral action: removing the error a steady load leaves . . . 12
   - 5.1 The dragging gearbox . . . 12
   - 5.2 Add the error up . . . 13
   - 5.3 Too much integral . . . 15
6. The three parts together: past, present and future . . . 16
7. Where PID runs: ROS 2, PX4, legged robots and Autoware . . . 18
8. Summary . . . 19
9. Sources . . . 20
10. Key terms . . . 20

Where this fits:

- **Builds on:** Derivatives of one variable; Open-loop and closed-loop control.
- **Leads to:** Second-order systems: natural frequency and damping ratio; Step response and its specifications; Root locus and Routh–Hurwitz; Cascade control; Integral action and the internal model principle; Integrator windup and anti-windup; Motion control and force control; PID forms; Discretising a controller; Sampling and the sample rate; Driving to a point.

1. Overview

Key point: A torque proportional to the heading error makes the robot swing past its new heading, because it arrives still turning. Adding a part that pushes against how fast the error is changing (D) brakes the turn in time; adding a part that grows with the error's running sum (I) removes the error a steady load leaves. Together they are the PID controller.

The robot can hold a speed: the closed loop measures its speed and pushes 20 N harder for every m/s it is too slow, and on a ramp it holds 0.464 m/s where a fixed push gives 0.108 m/s. Its route also has corners. At one corner the next leg of the route runs at 45°, so the robot must turn its **heading** (G-2284), the direction it faces, from 0° to 45° while it drives on at 0.5 m/s.

Turning takes a twist from the wheels, a **torque** (G-2808). The obvious rule is the one that worked for the speed: twist in proportion to the heading error, here 0.2 N m for every radian the heading is short of 45°. Figure 1 tries it. Watch the heading in the lower panel: it reaches 45° after 1.11 s, carries on to 90.5° at 2.22 s, comes back to −1.0° at 4.44 s and climbs again, so the robot snakes about the new leg.

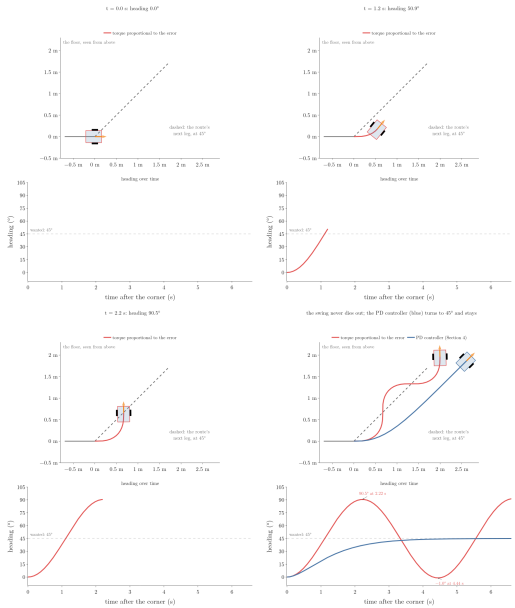


Figure 1: The robot reaches a corner of its route at 0.5 m/s and must turn from 0° to 45°; its torque is 0.2 N m per radian of heading error. Top: the floor seen from above, with the route's next leg dashed. Bottom: the heading over time. One frame per 0.2 s for 6.6 s: the heading swings to 90.5° at 2.22 s, back to −1.0° at 4.44 s, and up again. The last frame adds the PD controller of Section 4 (blue), which turns to 45° and stays

A second fault appears once the swing is cured: when one gearbox drags, a turn that should end at 45° stops at 39.27°, 5.73° short (Section 5). This Note fixes both, one part of the controller at a time:

- why turning swings when driving does not: force, torque and the moment of inertia (Section 2);
- proportional control, and why it overshoots (Section 3);
- derivative action, which brakes before the target (Section 4);
- integral action, which removes the error a steady load leaves (Section 5);
- the three parts together, and what each costs (Section 6);
- where PID controllers run in real robots (Section 7).

2. Why turning swings: force, torque and the moment of inertia

> **Key point:** A force on a mass sets its acceleration; a torque on a body sets its angular acceleration, by torque = moment of inertia × angular acceleration. The speed is one adding-up away from the force, but the heading is two adding-ups away from the torque, so a heading can arrive at its target still turning.

2.1 Driving: force sets the acceleration

The proportional rule held the speed without swinging and made the heading swing. To see why, we compare what the motors set in each case. For driving, **Newton's second law** (G-2809) says the net force on the robot equals its mass times its acceleration (MR §8.1.1). With all of the motors' largest push, 4 N, on the robot's 8 kg, from rest:

$$a = 4/8 = 0.5 \text{ m/s}^2$$

The acceleration then adds up over time into the speed: 0.5 m/s^2 for 1 s adds 0.5 m/s. One adding-up separates the push from the speed we control.

2.2 Turning: torque, angular acceleration and the moment of inertia

Torque. A turn is driven by the turning effect of the wheels' forces, their **torque** (G-2808): a force times its distance from the turning point, measured square to the force (MR §3.4). Each wheel is 0.1 m from the robot's centre. When the right wheel pushes 1 N forward and the left wheel 1 N back, both turn the robot to the left:

$$\tau = 1 \times 0.1 + 1 \times 0.1 = 0.2 \text{ N m}$$

Then 0.2 N m is the largest torque the robot uses for turning. The torque sets how fast the turn rate, the **angular velocity** (G-2292) ω, changes. That rate of change is the **angular acceleration** (G-2956), in rad/s^2 (OpenStax §10.7).

From force to torque. Take one small weight on a light rod, pushed square to the rod and turning about the rod's other end (OpenStax §10.7). With our numbers first:

- m is the weight's mass, 1 kg;
- r is its distance from the turning point, 0.2 m;
- F is the push, 1 N, square to the rod;
- a is the weight's acceleration along its circle, in m/s^2;
- α is the rod's angular acceleration, in rad/s^2;
- τ is the push's torque, F times r, 0.2 N m.

Newton's second law gives the weight's acceleration. A point at distance r from the centre travels r times the angle turned, the arc rule, so its acceleration along the circle is r times the angular acceleration. Then both sides are multiplied by r:

$$1 = 1 \times a$$

$$a = 0.2\,\alpha$$

$$1 = 1 \times 0.2\,\alpha$$

$$1 \times 0.2 = 1 \times 0.2^2\,\alpha$$

$$0.2 = 0.04\,\alpha$$

$$\alpha = 5 \text{ rad/s}^2$$

The same steps in symbols:

$$F = m\,a$$

$$a = r\,\alpha$$

$$F = m\,r\,\alpha$$

$$F\,r = m\,r^2\,\alpha$$

$$\tau = m\,r^2\,\alpha$$

The moment of inertia. A whole body is many such weights, each at its own distance. Adding up m times r^2 over all of them gives the body's **moment of inertia** (G-2955) I: how hard the body is to spin up or slow down about an axis, as mass says how hard it is to speed up (OpenStax §10.7). Mass far from the axis counts four times as much at twice the distance. Most of the robot's 8 kg, its battery and motors, sits near its centre, and its moment of inertia about the vertical axis is 0.1 kg m^2, as if all 8 kg sat 0.1118 m from the centre:

$$8 \times 0.1118^2 = 0.100 \text{ kg m}^2$$

For a whole body the weights' torques add up, and so do their m times r^2, which gives the rotational form of Newton's second law, **Newton's second law for rotation** (G-2957) (OpenStax §10.7, eq. 10.25), with:

- τ is the net torque on the robot, in N m, at most 0.2 N m;
- I is its moment of inertia, 0.1 kg m^2;
- α is its angular acceleration, in rad/s^2.

$$\tau = I\,\alpha$$

$$\alpha = 0.2/0.1 = 2 \text{ rad/s}^2$$

Figure 2 turns two robots with the same torque, 0.2 N m, for 1 s. The second carries two 1.25 kg weights 0.2 m either side of its centre, which add to its moment of inertia:

$$2 \times 1.25 \times 0.2^2 = 0.1 \text{ kg m}^2$$

The weights double it, to 0.2 kg m^2. Watch the dotted arcs: the loaded robot turns half as far.

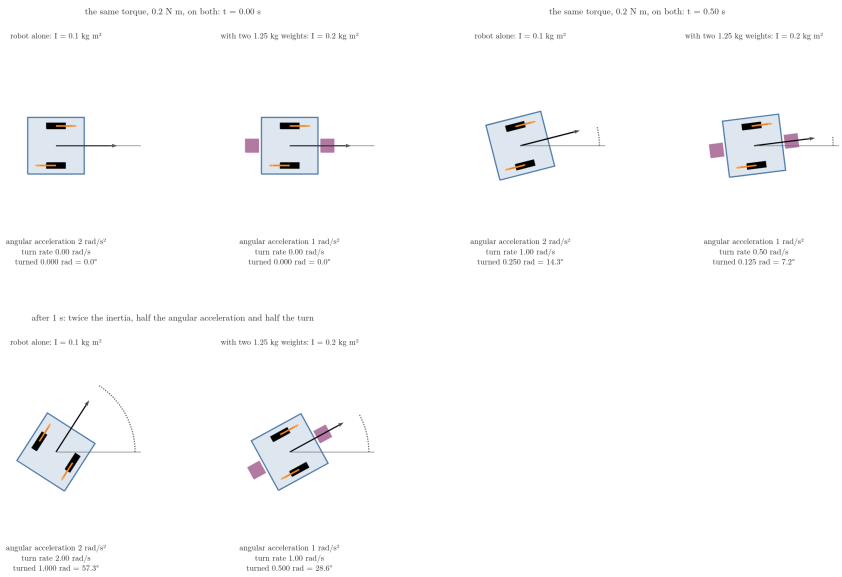


Figure 2: Two robots seen from above, each turned from rest by the same torque, 0.2 N m (orange arrows: right wheel 1 N forward, left wheel 1 N back), one frame per 0.05 s for 1 s. Left: the robot alone, moment of inertia 0.1 kg m^2: angular acceleration 2 rad/s^2; after 1 s it turns at 2 rad/s and has turned 1 rad (57.3°). Right: with two 1.25 kg weights (purple) 0.2 m either side of the centre, 0.2 kg m^2: 1 rad/s^2, 1 rad/s and 0.5 rad (28.6°). The dotted arc is the angle turned so far

Torque sets the angular acceleration just as force sets the acceleration. The difference lies in what we control: the next section shows what one more step between the motors and the heading does.

2.3 Why the heading swings: one adding-up more

Now the comparison. For the speed, force sets the acceleration, which adds up into the speed. For the heading, torque sets the angular acceleration, which adds up into the turn rate, which adds up again into the heading (Figure 3). With the heading error at 0, a proportional rule gives no torque, but the turn rate has been building up for the whole turn, so the robot turns on.

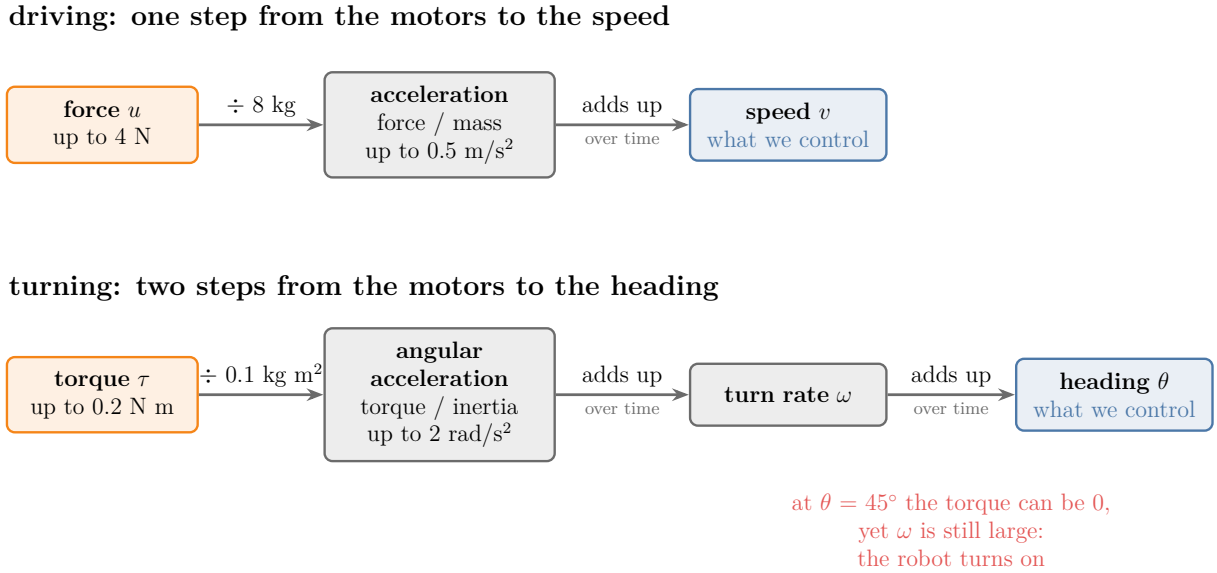


Figure 3: Driving (top): the force, up to 4 N, divided by the mass of 8 kg gives the acceleration, up to 0.5 m/s^2, which adds up over time into the speed, the quantity we control. Turning (bottom): the torque, up to 0.2 N m, divided by the moment of inertia of 0.1 kg m^2 gives the angular acceleration, up to 2 rad/s^2, which adds up into the turn rate, which adds up again into the heading, the quantity we control. At a heading of 45° the torque can be 0 while the turn rate is still large, so the robot turns on

The speed loop had a second advantage: the rolling drag grows with speed, so it absorbs any excess. Our heading model has no such drag: it leaves out the tyres' resistance to turning, so nothing absorbs an excess turn rate. The heading model, with:

- θ is the heading, in rad, 0 at the corner;
- ω is the turn rate, in rad/s, 0 at the corner;
- τ is the torque from the wheels, in N m, between -0.2 and 0.2 N m;
- I is the moment of inertia, 0.1 kg m^2;
- $\dot{\omega}$ and $\dot{\theta}$ are their rates of change (the dot notation (G-2296)).

$$I\,\dot{\omega} = \tau$$

$$\dot{\theta} = \omega$$

The computer sets a new torque every 10 ms and the motors hold it until the next. The next section puts the proportional rule into this model.

3. Proportional control: twisting in proportion to the error

> **Key point:** A torque proportional to the heading error pulls the heading towards 45° like a spring. At 45° the torque is 0 but the robot turns at 1.12 rad/s, so it swings on to 90.5° and back; with nothing to absorb the motion, the swing never dies out.

3.1 The rule, and why it overshoots

The rule of Figure 1 is the speed loop's rule moved to the heading: the further the heading is from where it should be, the harder the twist. Setting the command in proportion to the control error is **proportional control** (G-2958), P control for short, and the factor is the **proportional gain** (G-2959) K_p (FBS §1.6; MR §11.3). The speed loop's 20 N per m/s of error was a proportional gain too. For the heading, with:

- θ_r is the wanted heading, the **reference** (G-2944): 45°, which is 0.785 rad;
- θ is the measured heading, in rad;
- e is the heading error, the **control error** (G-2943), in rad;
- K_p is the proportional gain, 0.2 N m per rad;
- τ is the torque the controller asks for, in N m.

$$e = \theta_r - \theta$$

$$\tau = K_p\, e$$

At the corner the whole 45° is missing:

$$e = 0.785 - 0 = 0.785 \text{ rad}$$

$$\tau = 0.2 \times 0.785 = 0.157 \text{ N m}$$

The 0.157 N m is within the 0.2 N m the wheels can give. Figure 4 shows the turn with three panels sharing one time axis: the heading, the turn rate and the torque. Watch the moment the heading reaches 45°, at 1.11 s, marked by the dotted line.

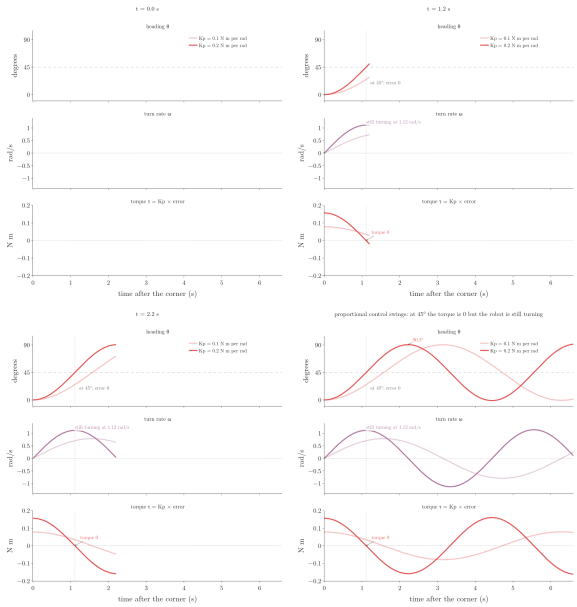


Figure 4: Proportional control of the heading, Kp = 0.2 N m per rad (bold) and 0.1 (faint); one frame per 0.2 s for 6.6 s. Top: heading; middle: turn rate; bottom: torque, Kp times the error. At 1.11 s the heading reaches 45°: the error and the torque are 0, but the robot turns at 1.12 rad/s, so it carries on to 90.5°, where the torque, now negative, has stopped it and sends it back. The smaller gain swings more slowly (peak at 3.14 s instead of 2.22 s) but just as far

At 1.11 s the torque is 0 and the robot turns at 1.12 rad/s. Past 45° the error is negative, so the torque pushes back, but it starts from 0 and needs time to stop the turn: the robot stops only at 90.5°, at 2.22 s. Then the same happens the other way, down to −1.0° at 4.44 s.

3.2 A spring with nothing to stop it

The proportional rule acts like a spring between the robot's heading and the 45° line: the further the heading is pulled away, the harder it pulls back, 0.2 N m per rad. MR §11.2 puts it the same way: "designing a controller is much like choosing the spring constant" and the damping of a mass on a spring. A weight on a spring with nothing to slow it bounces for ever, and so does the heading here: with no friction on turning, nothing takes the motion out, and the swing does not die out.

A different gain does not help. With 0.1 N m per rad (faint in Figure 4) the pull is weaker, so the robot turns more slowly and swings more slowly, its first peak at 3.14 s instead of 2.22 s, but just as far. A stronger spring swings faster, not less.

The swing comes from the turn rate the robot builds up, which the proportional rule never looks at. The speed loop of feedback basics settled under the same kind of rule because rolling drag damped it; a proportional controller swings for ever only when nothing damps the motion, as here, with no friction on turning. The fix is to look at how fast the error is changing.

4. Derivative action: braking before the target

> **Key point:** Adding a torque that pushes against how fast the error is changing brakes the turn while the robot is still short of 45°. With a derivative gain of 0.283 N m per rad/s the heading reaches 45° without passing it.

4.1 Look at how fast the error shrinks

Figure 1's swing happens because the robot reaches 45° still turning at 1.12 rad/s. A driver steering back into a lane does not wait for the lane line to start straightening the wheel: seeing the car close in fast, they counter-steer early. The controller can do the same if it looks at how fast the error is shrinking, the error's rate of change, its **derivative** (G-595) with respect to time. A shrinking error means the robot is already moving towards 45°, so the controller should push less, or even push back.

The computer measures the heading every 10 ms, so it estimates the derivative from two readings in a row: the change in the error divided by the 0.01 s between them. With the wanted heading fixed at 45°, the error shrinks exactly as fast as the heading grows, so this rate of change is minus the turn rate.

A torque in proportion to the error's rate of change is **derivative action** (G-2960), with the **derivative gain** (G-2961) K_d (FBS §11.1). A controller with a proportional part and a derivative part is a **PD controller** (G-2962) (MR §11.4). With the symbols of Section 3.1, the heading error e and the proportional gain K_p, 0.2 N m per rad, and:

- de/dt is the error's rate of change, in rad/s, estimated as the change since the last reading divided by 0.01 s;
- K_d is the derivative gain, 0.283 N m per rad/s.

$$\tau = K_p\, e + K_d\, \frac{de}{dt}$$

At 1.0 s into the turn (notebook), the PD robot's heading is 18.78°, its error 0.4576 rad, and the error shrinks at 0.3827 rad/s:

$$\text{P part} = 0.2 \times 0.4576 = 0.0915$$

$$\text{D part} = 0.283 \times (-0.3827)$$

$$= -0.1083$$

$$\tau = 0.0915 - 0.1083 = -0.0168 \text{ N m}$$

Still 26° short of 45°, the PD robot already brakes, and turns at 0.38 rad/s. At the same moment the P-only robot of Section 3 is at 38.1°, turning at 1.10 rad/s and still pushing with 0.024 N m. The derivative part acts like a shock absorber, a damper whose force grows with speed: MR §11.4 notes that K_d "plays the same role as the viscous friction" that the turn lacks.

4.2 How much derivative

Figure 5 keeps K_p at 0.2 N m per rad and raises K_d one value per frame, redrawing the turn each time. The lower panel shows the two parts of the torque for the bold curve: the P part (red) pulls towards 45°, the D part (purple) pushes against the turning.

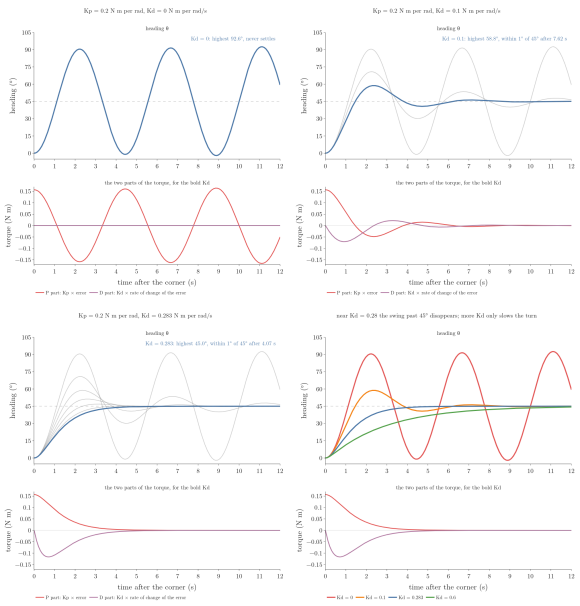


Figure 5: The derivative gain swept with Kp = 0.2 N m per rad: one Kd per frame, from 0 to 0.6 N m per rad/s, the heading over 12 s for the current Kd (bold) and the earlier ones (faint). Below: the P part (red) and D part (purple) of the torque for the current Kd. Kd = 0 swings to 92.6° within 12 s; 0.1 overshoots to 58.8°; 0.283 reaches 45.0° without passing it and stays within 1° after 4.07 s; 0.6 creeps up and needs 10.94 s. The last frame holds these four

What the sweep shows (notebook):

K_d (N m per rad/s)	Highest heading	Within 1° of 45° for good after
0	92.6° within 12 s	never
0.1	58.8°	7.62 s
0.283	45.0°	4.07 s
0.6	44.8°	10.94 s

Too little derivative leaves a swing; too much makes the turn sluggish, because the brake works against the turn from the start. Near 0.28 the swing past 45° just disappears, and we use 0.283 N m per rad/s; reading a controller's response shows where that value comes from.

A gotcha: the derivative of a noisy reading. Dividing a change by 0.01 s magnifies every small error in the readings. If each heading reading jitters by 0.005 rad (0.29°), the differenced error jitters by about 0.71 rad/s, and the D part by about 0.2 N m, the robot's whole torque (notebook):

$$\sqrt{2} \times 0.005/0.01 = 0.71 \text{ rad/s}$$

$$0.283 \times 0.71 = 0.2 \text{ N m}$$

FBS §11.1 warns of this: prediction by "linear extrapolation can generate large undesired control signals because measurement noise is amplified". Our robot has a gyro, which measures the turn rate directly, so its D part can use minus the gyro's reading instead of differencing the heading; Section 7 shows real controllers doing exactly that. Filtering the derivative is part of PID tuning in practice.

The PD controller turns the robot to 45° without a swing, where the proportional rule alone swung it to 90.5°, as long as nothing pushes the robot off course. The next section adds such a push.

5. Integral action: removing the error a steady load leaves

> **Key point:** A gearbox that drags leaves the PD robot 5.73° short, because only an error can produce the torque that fights the drag. Adding the error up over time and twisting in proportion to the sum removes the error: the sum keeps growing until it alone supplies that torque.

5.1 The dragging gearbox

The running example changes here: the robot's right gearbox has worn and drags, a steady torque of 0.02 N m turning the robot to the right, whatever it does. The drag pushes back on the right side of the robot, 0.1 m from its centre, and by the torque rule a backward push on the right turns the robot to the right, as a shopping trolley with a stiff right wheel pulls to the right. Such a steady load is a **disturbance** (G-2949) on the plant, like the ramp of the speed loop. The heading model of Section 2.3 gains one term, with:

- τ_d is the drag's torque, -0.02 N m (negative: to the right).

$$I\,\dot{\omega} = \tau + \tau_d$$

The PD robot of Section 4 now stops at 39.27°, 5.73° short (Figure 6, blue). Once it has settled it no longer turns, so the turn rate and the error's rate of change are both 0, and the torques must balance: the P part alone holds off the drag. Numbers first:

$$0 = 0.2\,e - 0.02$$

$$e = 0.02/0.2 = 0.1 \text{ rad}$$

$$= 5.73°$$

In symbols, with the proportional gain K_p:

$$0 = K_p\,e + \tau_d$$

$$e = -\tau_d/K_p$$

The error left once everything has settled, the steady-state error, is not 0, for the same reason the speed loop kept 0.036 m/s on the ramp: the controller gives the extra torque only through an error (FBS §1.6). A larger K_p shrinks the error but never removes it, and the derivative part cannot help, since it is 0 once the robot has stopped turning.

5.2 Add the error up

A driver whose car pulls to one side notices, after a while, that holding the wheel straight keeps leaving them off course, and holds it a little to the other side. The controller can do the same by keeping a running sum of the error: every 10 ms it adds the error times 0.01 s. While an error persists, the sum keeps growing, and so does the torque it gives. For example, three readings in a row with an error of 0.1 rad add:

$$0.1 \times 0.01 + 0.1 \times 0.01 + 0.1 \times 0.01$$

$$= 0.003 \text{ rad s}$$

As the readings come closer together, this running sum becomes the area under the curve of the error against time, its integral, written $\int_0^t e\,dt$: the sign $\int$ means "add up, step by step, from time 0 to now". A torque in proportion to the error's running sum is **integral action** (G-2963), with the **integral gain** (G-2964) K_i (FBS §1.6). Together with the P and D parts it makes a **PID controller** (G-2965) (FBS §11.1, eq. 11.1). With the symbols of Section 4.1, the error e, its rate of change de/dt and the gains K_p, 0.2 N m per rad, and K_d, 0.283 N m per rad/s, and:

- K_i is the integral gain, 0.05 N m per rad s;
- $\int_0^t e\,dt$ is the error's running sum from the corner to now, in rad s.

$$\tau = K_p\, e + K_i \int_0^t e\,dt + K_d\, \frac{de}{dt}$$

Why it ends with no error. Suppose the robot settles, its heading no longer changing. Then the running sum must stop changing too, and a sum stops changing only if each new addition, the error times 0.01 s, is 0. So whenever a PID controller settles, it settles with no error, whatever the drag (FBS §11.1, "the magic of integral action"). The sum ends wherever its torque cancels the drag:

$$0.05 \times \text{sum} = 0.02$$

$$\text{sum} = 0.02/0.05 = 0.4 \text{ rad s}$$

Figure 6 runs the PD and PID robots through the corner with the dragging gearbox. Watch the green area in the bottom panel, the integral part, built up one 10 ms step at a time, against the dashed line at the 0.02 N m it must supply.

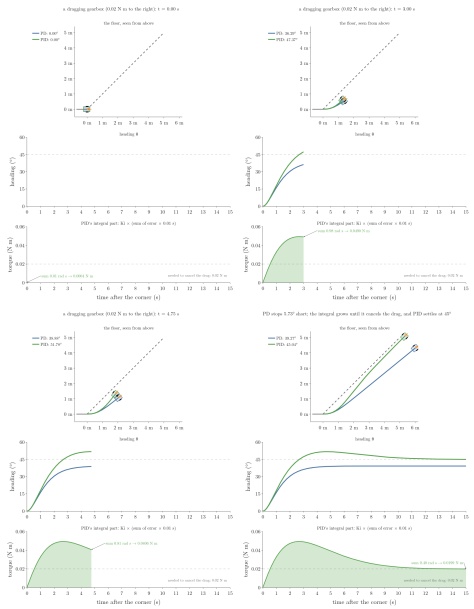


Figure 6: The dragging gearbox, 0.02 N m to the right, on two robots at the corner: PD (Kp 0.2, Kd 0.283; blue) and PID (the same plus Ki 0.05; green); one frame per 0.25 s for 15 s. Top: the floor seen from above, the route's next leg dashed. Middle: the heading. Bottom: the PID's integral part, Ki times the running sum of the error, against the 0.02 N m needed to cancel the drag. PD stops at 39.27°; the PID's sum grows to 0.98 rad s by 3 s (0.049 N m), the heading passes 45° to 51.78° at 4.74 s, and the sum shrinks back until the integral part is 0.02 N m and the heading 45°

The sum grows fastest at the start, while the error is large: by 3 s it has reached 0.98 rad s, far more than the 0.4 rad s it will need. To shrink it back, the error has to turn negative, so the heading passes 45°, up to 51.78° at 4.74 s. Then the sum unwinds, and the heading settles at 45°, where the PD robot stopped at 39.27°, with the integral part at 0.02 N m: the integral part alone now holds off the drag, with no error left.

5.3 Too much integral

The integral removes the error, but it also adds back the swing past 45° that the derivative had removed: even with no drag, the PID turn peaks at 55.1° (notebook), because the sum grows during the whole turn. Figure 7 keeps K_p and K_d and raises K_i one value per frame.

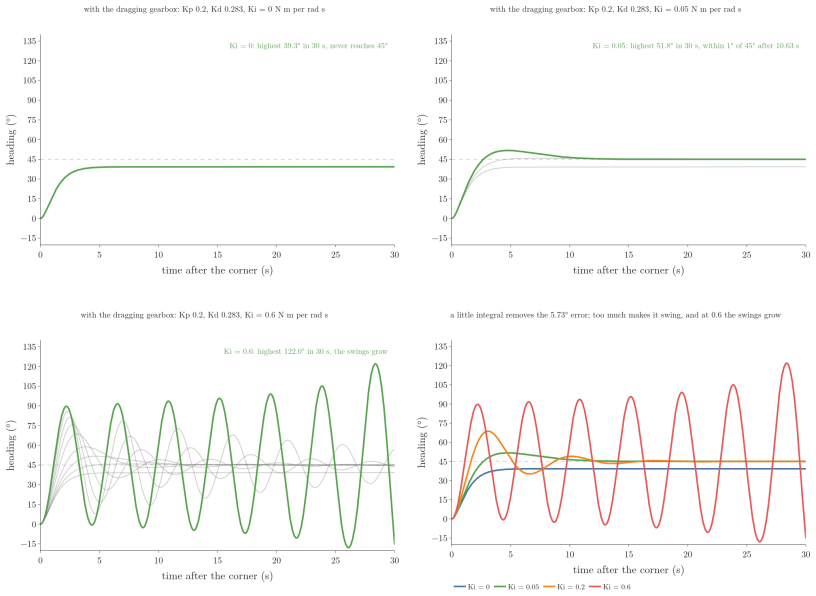


Figure 7: The integral gain swept, with the dragging gearbox, Kp 0.2 and Kd 0.283: one Ki per frame from 0 to 0.6 N m per rad s, the heading over 30 s for the current Ki (bold) and the earlier ones (faint). Ki = 0 stops at 39.3°; 0.05 peaks at 51.8° and is within 1° of 45° after 10.63 s; larger values swing more; at 0.6 every swing is bigger than the last. The last frame holds Ki = 0, 0.05, 0.2 and 0.6

What the sweep shows (notebook):

K_i (N m per rad s)	Highest heading	Within 1° of 45° for good after
0	39.27°	never
0.05	51.78°	10.63 s
0.2	68.73°	14.77 s
0.4	81.28°	38.51 s
0.6	89.8°, 91.7°, 93.6°, 95.8°, …	never

A larger integral gain makes the sum act sooner, but it also overshoots harder and settles more slowly. At 0.6 the swings grow without end: the controller has become unstable, meaning that after a disturbance the heading moves ever further from its target instead of coming back to rest. MR §11.4 makes the same point: the integral gain has an upper limit for stability, and "the integral term can improve steady-state response, but it can worsen the transient response". So robot controllers often keep K_i small, or 0, and hold the integral part inside fixed bounds (MR §11.4). Holding it inside bounds is called limiting the integral term, one of the fixes for what goes wrong when the torque is at its limit.

The same integral also cures the speed loop's ramp error: adding an integral gain of 10 N per metre of summed error (the speed error's running sum, in m/s times s) to the speed loop of feedback basics, the speed dips to 0.472 m/s on the ramp and returns to 0.500 m/s (notebook), instead of staying at 0.464 m/s.

6. The three parts together: past, present and future

> **Key point:** P looks at the error now, I at its past, D at where it is heading. Each fixes one fault of the others and brings one cost.

The three parts read the same error curve in three ways (FBS §1.6, Fig. 1.15). Figure 8 shows the heading error of the PID run of Figure 6 and marks, at 1 s into the turn, what each part reads:

- the present: the error now, the red dot;
- the past: the area under the error curve so far, shaded green;
- the future: the slope of the curve now, the purple dashed line, which extends the error 0.5 s ahead.

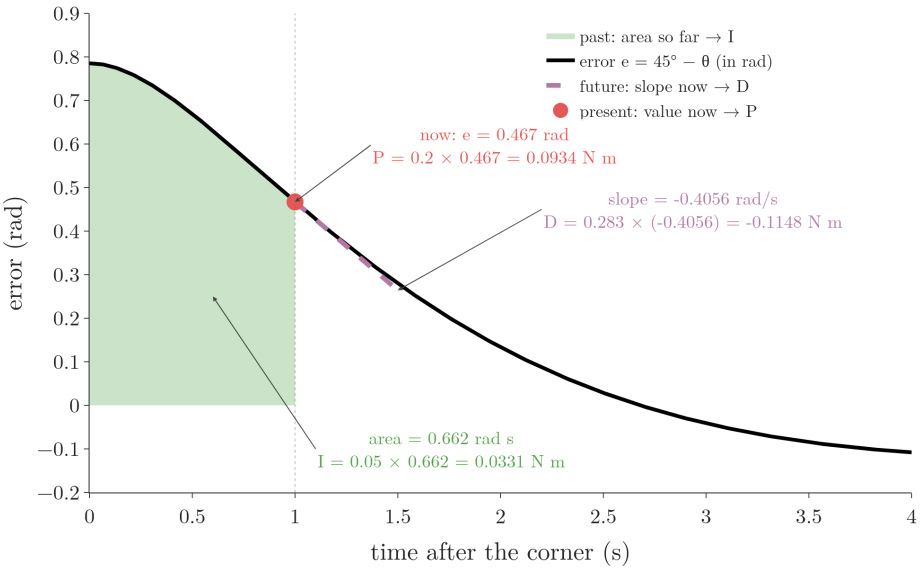


Figure 8: The heading error of the PID run with the dragging gearbox, from the corner to 4 s. At 1 s: the error now, 0.467 rad, gives the P part, $0.2 \times 0.467 = 0.0934$ N m; the area under the curve so far, 0.662 rad s, gives the I part, $0.05 \times 0.662 = 0.0331$ N m; the slope now, -0.4056 rad/s, gives the D part, $0.283 \times (-0.4056) = -0.1148$ N m

At 1 s the three parts add up to the torque the controller asks for:

$$0.0934 + 0.0331 - 0.1148$$

$$= 0.0117 \text{ N m}$$

Part	Reads	Fixes	Costs
P, $K_p\, e$	the error now	pulls the heading towards its target	alone, swings when nothing damps the motion, and leaves an error under a steady load
D, $K_d\, de/dt$	how fast the error changes	brakes early, removing the swing	passes noise on, magnified by the 0.01 s division
I, $K_i \int e\, dt$	the error's running sum	removes the error a steady load leaves	adds a swing; too large a gain makes the loop unstable

Not every loop needs all three. A controller with no integral part is a PD controller, common for robot joints (MR §11.4); one with no derivative part is a PI controller, the commonest

kind in industry, where "more than 95% of all industrial control problems are solved by PID control, although many of these controllers are actually proportional-integral (PI) controllers" (FBS §1.6). The speed loop of feedback basics needs no D part, because drag already damps it; the heading needs D, because nothing else does.

7. Where PID runs: ROS 2, PX4, legged robots and Autoware

The PID controller of Section 5 runs, nearly line for line, in the stacks robots use. The code below was read for this Note.

ros2_control. The PID used by `ros2_control`'s `pid_controller` is `Pid::compute_command` in `control_toolbox/src/pid.cpp` (repository `ros-controls/control_toolbox`). Its steps are those of Section 4.1 and Section 5.2:

- the derivative from two readings: `d_error_ = (error - p_error_last_) / dt_s;`
- the P part: `p_term = gains_.p_gain_ * p_error_;`
- the D part: `d_term = gains_.d_gain_ * d_error_;`
- the integral part, the running sum: `i_term_ += dt_s * gains_.i_gain_ * error;`
- the command: `cmd_unsat_ = p_term + i_term_ + d_term;`, then held between the limits `u_min_` and `u_max_`, with the integral part held between `i_min` and `i_max`, which is limiting the integral term.

PX4. The flight software of many drones turns each axis with a PID on the turn rate, in `src/lib/rate_control/rate_control.cpp` (repository `PX4/PX4-Autopilot`): `torque = _gain_p.emult(rate_error) + _rate_int - _gain_d.emult(angular_accel) + _gain_ff.emult(rate_sp);`. Two details match this Note:

- its D part uses the measured angular acceleration, not a differenced error, the gotcha of Section 4.2;
- the yaw-rate gains in `src/modules/mc_rate_control/mc_rate_control_params.yaml` default to `MC_YAWRATE_P` 0.2, `MC_YAWRATE_I` 0.1 and `MC_YAWRATE_D` 0.0, and the integral gain's description reads "Can be set to compensate static thrust difference or gravity center offset": a steady load like our dragging gearbox.

Legged robots trained by reinforcement learning. In `legged_gym` (repository `leggedrobotics/legged_gym`), the learned policy outputs a target angle for each joint, and a PD controller turns it into a torque, in `_compute_torques` of `legged_gym/envs/base/legged_robot.py`: `torques = self.p_gains*(actions_scaled + self.default_dof_pos - self.dof_pos) - self.d_gains*self.dof_vel`. Its D part is minus a gain times the measured joint speed, as our gyro would give. The A1 robot's settings in `legged_gym/envs/a1/a1_config.py` are a stiffness of 20 N m per rad and a damping of 0.5 N m per rad/s: K_p and K_d under other names.

Autoware. The longitudinal controller of feedback basics runs a PI on the speed error: `kp: 1.0`, `ki: 0.1`, `kd: 0.0` in its parameter file, with the integral term limited by `max_i_effort: 0.3`. Its own `PIDController::calculate` in `src/pid.cpp` sets the derivative to 0 on its first call, when there is no previous error, as our notebook does.

8. Summary

Controller	Torque	At the corner	With the dragging gearbox
P	$K_p\, e$	swings for ever: 90.5° at 2.22 s, back to −1.0° at 4.44 s	swings about 39.27°, up to 79.0°
PD	$K_p\, e + K_d\, de/dt$	45.0° without passing it, within 1° after 4.07 s	stops at 39.27°, 5.73° short
PID	$K_p\, e + K_i \int e\, dt + K_d\, de/dt$	45° after peaking at 55.1°, within 1° after 10.95 s	45° after peaking at 51.78°, within 1° after 10.63 s

With $K_p = 0.2$ N m per rad, $K_d = 0.283$ N m per rad/s and $K_i = 0.05$ N m per rad s; e is the heading error, 45° minus the heading, in rad.

- The opening question has its answer: twisting in proportion to the heading error swings the robot to 90.5° and back, because it reaches 45° turning at 1.12 rad/s; **derivative action** (G-2960) brakes the turn in time, and **integral action** (G-2963) removes the 5.73° a dragging gearbox leaves, so the **PID controller** (G-2965) turns to 45° and stays there.
- **Newton's second law for rotation** (G-2957), torque = **moment of inertia** (G-2955) × **angular acceleration** (G-2956), sets how fast a torque spins the robot up: 0.2 N m on 0.1 kg m^2 gives 2 rad/s^2, so doubling the moment of inertia halves the turn.
- The heading is two adding-ups away from the torque, the speed only one, so a heading can arrive at its target still turning while a speed cannot.
- **Proportional control** (G-2958) with **proportional gain** (G-2959) K_p acts like a spring, so without damping it swings for ever; a larger gain swings faster, not less.
- **PD control** (G-2962) adds a torque proportional to the error's rate of change with **derivative gain** (G-2961) K_d, which acts like a damper, so near 0.28 N m per rad/s the swing past 45° disappears; it magnifies sensor noise, so real controllers use a measured turn rate.
- A steady load leaves an error under P or PD, 0.1 rad for 0.02 N m of drag, because only an error can produce the torque that holds the load off.
- The running sum with **integral gain** (G-2964) K_i grows while any error remains, so a settled PID loop has no error; it overshoots, and too large a gain, 0.6 here, makes the loop unstable.

The robot can now turn to a heading and hold it against a steady drag. Still missing: words and numbers to judge a turn, how far it overshoots, how fast it settles and why the swing disappears near 0.28; reading a controller's response supplies them.

9. Sources

Built from

- Udacity, *Artificial Intelligence for Robotics* (CS373), Lesson 5, "PID Control", "Proportional Control", "Implement P Controller", "Oscillations", "PD Controller", "Systematic Bias", "Is PD Enough" and "PID Implementation", videos on the Udacity YouTube channel; transcript: https://coursedocs.readthedocs.io/en/latest/gatech/cs8803-O01/pid.html
- Northwestern Robotics, "Modern Robotics, Chapter 11.4: Motion Control with Torque or Force Inputs (Part 1 of 3)", YouTube, https://www.youtube.com/watch?v=HAVBtupm8zM
- Northwestern Robotics, "Modern Robotics, Chapter 11.4: Motion Control with Torque or Force Inputs (Part 2 of 3)", YouTube, https://www.youtube.com/watch?v=992hdsGgVlE
- Åström, K. J. and Murray, R. M. (2021). *Feedback Systems*, 2nd ed., §1.6 "Simple Forms of Feedback" (Fig. 1.15), §11.1 "Basic Control Functions" (eq. 11.1). Free: https://fbswiki.org (FBS)
- Lynch, K. M. and Park, F. C. (2017). *Modern Robotics: Mechanics, Planning, and Control*. Cambridge University Press. §3.4 torque, §8.1.1 Newton's second law, §11.2 error dynamics, §11.3 P control with velocity inputs, §11.4 PD and PID control with torque inputs. Free preprint: http://modernrobotics.org (MR)
- OpenStax, *University Physics Volume 1*, §10.7 "Newton's Second Law for Rotation" (eq. 10.25), https://openstax.org/books/university-physics-volume-1/pages/10-7-newtons-second-law-for-rotation (OpenStax §10.7)

Other references

- Northwestern Robotics, "Modern Robotics, Chapter 11.3: Motion Control with Velocity Inputs (Part 2 of 3)", YouTube, https://www.youtube.com/watch?v=NAXmXg1gCI0
- ros-controls, control_toolbox, the PID class and its compute_command function (file control_toolbox/src/pid.cpp), https://github.com/ros-controls/control_toolbox (read for Section 7)
- PX4, PX4-Autopilot, the multicopter rate controller (file src/lib/rate_control/rate_control.cpp) and its yaw-rate parameters (file src/modules/mc_rate_control/mc_rate_control_params.yaml), https://github.com/PX4/PX4-Autopilot (read for Section 7)
- Legged Robotics (ETH Zurich), legged_gym, the torque computation of the base robot class and the A1 robot's settings (files legged_gym/envs/base/legged_robot.py and legged_gym/envs/a1/a1_config.py), https://github.com/leggedrobotics/legged_gym (read for Section 7)
- Autoware Foundation, autoware_universe, the PID longitudinal controller's parameter file and PID class (the controller's folder under control/), https://github.com/autowarefoundation/auto (read for Section 7)

10. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Angular acceleration (G-2956)	How fast the turn rate changes, in rad/s^2, such as 2 rad/s^2 when 0.2 N m turns a body of 0.1 kg m^2; a torque sets it, and it adds up over time into the turn rate.
Moment of inertia (G-2955)	How hard a body is to spin up or slow down about an axis: the sum of each bit of mass times its distance from the axis squared, such as 0.1 kg m^2 for the robot about its vertical axis; it plays for turning the part mass plays for driving.
Newton's second law for rotation (G-2957)	The net torque on a body equals its moment of inertia times its angular acceleration, tau = I alpha; it gives how fast a torque spins a body up, the turning form of F = ma.
Proportional control (P control) (G-2958)	Feedback that sets the command in proportion to the control error, such as a torque of 0.2 N m per rad of heading error; it pulls the output towards the reference like a spring, but alone it swings when nothing damps the motion and leaves an error under a steady load.
Proportional gain (G-2959)	The factor K_p by which a proportional controller multiplies the error, such as 0.2 N m per rad or 20 N per m/s; a larger gain corrects harder, and for an undamped plant swings faster.
Derivative action (G-2960)	A part of the command in proportion to the error's rate of change, estimated from two readings in a row; it pushes against fast changes, so it brakes a turn before the target and damps swings, but it magnifies sensor noise.
Derivative gain (G-2961)	The factor K_d by which derivative action multiplies the error's rate of change, such as 0.283 N m per rad/s; too little leaves a swing, too much makes the response sluggish.
PD control (G-2962)	Feedback with a proportional part and a derivative part, such as K_p e + K_d de/dt; it acts like a spring and a damper, so it can bring a frictionless joint or heading to its target without swinging.

Term	Meaning
Integral action (G-2963)	A part of the command in proportion to the error's running sum (its integral over time); the sum grows while any error remains, so it removes the error a steady load leaves, at the cost of overshoot and, if too large, instability.
Integral gain (G-2964)	The factor K_i by which integral action multiplies the error's running sum, such as 0.05 N m per rad s; larger values remove a steady error sooner but swing more, and above a limit make the loop unstable.
PID controller (G-2965)	A controller whose command is the sum of a proportional, an integral and a derivative part of the error, K_p e + K_i (integral of e) + K_d de/dt; the commonest feedback controller, it reacts to the error now, its past and its trend; with K_d = 0 it is a PI controller, with K_i = 0 a PD controller.
Heading θ (G-2284)	The angle from the world x-axis to the direction a robot faces, measured counter-clockwise; it decides which way the robot moves when it drives forward.
Torque (G-2808)	The turning effect of a force about a point: the force times its distance from the point, measured square to the force's line; comparing torques decides whether a pushed object tips.
Newton's second law (G-2809)	The net force on a body equals its mass times its acceleration; it gives the force needed to change an object's speed, which the quasistatic assumption neglects.
Angular velocity ω (G-2292)	How fast a robot's heading changes, in rad/s, positive to the left; for a differential drive, the difference of the wheel speeds divided by the wheel separation.
Dot notation $\dot{x}$ (G-2296)	A dot over a quantity means its rate of change per second (its derivative with respect to time), such as $\dot{x} = 0.433$ m/s; models of motion are written with it.
Reference (G-2944)	The value we want the controlled output to have, such as a wanted speed of 0.5 m/s from the plan; the controller compares the measured output with it.

Term	Meaning
Control error (G-2943)	The reference minus the measured output, with its sign, such as $0.5 - 0.45 = 0.05$ m/s; a feedback controller computes its command from it, so a correction appears only once there is an error.
Derivative (G-595)	The slope of a function at a point: how fast the output changes as the input changes there. It is zero at the lowest point of a smooth curve, which is how training finds a minimum.
Disturbance (G-2949)	An unwanted outside effect that changes what the plant does, such as a ramp pulling a robot back with 0.785 N; it changes the true output, unlike sensor noise, which changes only the reading.